

6.5840 Lab 4: Fault-tolerant Key/Value Service

[Collaboration policy](#) // [Submit lab](#) // [Setup Go](#) // [Guidance](#) // [Piazza](#)

Introduction

In this lab you will build a fault-tolerant key/value storage service using your Raft library from [Lab 3](#). To clients, the service looks similar to the server of [Lab 2](#). However, instead of a single server, the service consists of a set of servers that use Raft to help them maintain identical databases. Your key/value service should continue to process client requests as long as a majority of the servers are alive and can communicate, in spite of other failures or network partitions. After Lab 4, you will have implemented all parts (Clerk, Service, and Raft) shown in the [diagram of Raft interactions](#).

Clients will interact with your key/value service through a Clerk, as in Lab 2. A Clerk implements the Put and Get methods with the same semantics as Lab 2: Puts are at-most-once and the Puts/Gets must form a linearizable history.

Providing linearizability is relatively easy for a single server. It is harder if the service is replicated, since all servers must choose the same execution order for concurrent requests, must avoid replying to clients using state that isn't up to date, and must recover their state after a failure in a way that preserves all acknowledged client updates.

This lab has three parts. In part A, you will implement a replicated-state machine package, rsm, using your raft implementation; rsm is agnostic of the requests that it replicates. In part B, you will implement a replicated key/value service using rsm, but without using snapshots. In part C, you will use your snapshot implementation from Lab 3D, which will allow Raft to discard old log entries. Please submit each part by the respective deadline.

You should review the [extended Raft paper](#), in particular Section 7 (but not 8). For a wider perspective, have a look at Chubby, Paxos Made Live, Spanner, Zookeeper, Harp, Viewstamped Replication, and [Bolosky et al.](#)

Start early.

Getting Started

We supply you with skeleton code and tests in src/kvraft1. The skeleton code uses the skeleton package src/kvraft1/rsm to replicate a server. A server must implement the StateMachine interface defined in rsm to replicate itself using rsm. Most of your work will be implementing rsm to provide server-agnostic replication. You will also need to modify kvraft1/client.go and kvraft1/server.go to implement the server-specific parts. This split allows you to re-use rsm in the next lab. You may be able to re-use some of your Lab 2 code (e.g., re-using the server code by copying or importing the "src/kvsrv1" package) but it is not a requirement.

To get up and running, execute the following commands. Don't forget the git pull to get the latest software.

```
$ cd ~/6.5840
$ git pull
..
```

Part A: replicated state machine (RSM)

```
$ cd src
$ make rsm1
=== RUN    TestBasic4A
Test RSM basic (reliable network)...
    rsm_test.go:28: expected 0 instead of 0
```

In the common situation of a client/server service using Raft for replication, the service interacts with Raft in two ways: the service leader submits client operations by calling `raft.Start()`, and all service replicas receive committed operations via Raft's `applyCh`, which they execute. On the leader, these two activities interact. At any given time, some server goroutines are handling client requests, have called `raft.Start()`, and each is waiting for its operation to commit and to find out what the result of executing the operation is. And as committed operations appear on the `applyCh`, each needs to be executed by the service, and the results need to be handed to the goroutine that called `raft.Start()` so that it can return the result to the client.

The `rsm` package encapsulates the above interaction. It sits as a layer between the service (e.g. a key/value database) and Raft. In `rsm/rsm.go` you will need to implement a "reader" goroutine that reads the `applyCh`, and a `rsm.Submit()` function that calls `raft.Start()` for a client operation and then waits for the reader goroutine to hand it the result of executing that operation.

The service that is using `rsm` appears to the `rsm` reader goroutine as a `StateMachine` object providing a `DoOp()` method. The reader goroutine should hand each committed operation to `DoOp()`; `DoOp()`'s return value should be given to the corresponding `rsm.Submit()` call for it to return. `DoOp()`'s argument and return value have type `any`; the actual values should have the same types as the argument and return values that the service passes to `rsm.Submit()`, respectively.

The service should pass each client operation to `rsm.Submit()`. To help the reader goroutine match `applyCh` messages with waiting calls to `rsm.Submit()`, `Submit()` should wrap each client operation in an `Op` structure along with a unique identifier. `Submit()` should then wait until the operation has committed and been executed, and return the result of execution (the value returned by `DoOp()`). If `raft.Start()` indicates that the current peer is not the Raft leader, `Submit()` should return an `rpc.ErrWrongLeader` error. `Submit()` should detect and handle the situation in which leadership changed just after it called `raft.Start()`, causing the operation to be lost (never committed).

For Part A, the `rsm` tester acts as the service, submitting operations that it interprets as increments on a state consisting of a single integer. In Part B you'll use `rsm` as part of a key/value service that implements `StateMachine` (and `DoOp()`), and calls `rsm.Submit()`.

If all goes well, the sequence of events for a client request is:

- The client sends a request to the service leader.
- The service leader calls `rsm.Submit()` with the request.
- `rsm.Submit()` calls `raft.Start()` with the request, and then waits.

- Raft commits the request and sends it on all peers' applyChs.
- The rsm reader goroutine on each peer reads the request from the applyCh and passes it to the service's DoOp().
- On the leader, the rsm reader goroutine hands the DoOp() return value to the Submit() goroutine that originally submitted the request, and Submit() returns that value.

Implement rsm.go: the Submit() method and a reader goroutine. You have completed this task if you pass the rsm 4A tests:

```
$ cd src
$ make RUN="-run 4A" rsm1
go build -race -o main/rsmdl main/rsmdl.go
cd kvraft1/rsm; go test -v -race -run 4A
=== RUN    TestBasic4A
Test RSM basic (reliable network)...
... Passed -- time 4.2s #peers 3 #RPCs 50 #Ops 10
--- PASS: TestBasic4A (4.57s)
=== RUN    TestConcurrent4A
Test concurrent submit (reliable network)...
... Passed -- time 1.0s #peers 3 #RPCs 28 #Ops 50
--- PASS: TestConcurrent4A (1.39s)
=== RUN    TestLeaderFailure4A
Test Leader Failure (reliable network)...
... Passed -- time 2.9s #peers 3 #RPCs 32 #Ops 2
--- PASS: TestLeaderFailure4A (3.29s)
=== RUN    TestLeaderPartition4A
Test Leader Partition (reliable network)...
2026/03/11 10:43:46 partition leader 0
... Passed -- time 3.6s #peers 3 #RPCs 61 #Ops 2
--- PASS: TestLeaderPartition4A (4.04s)
=== RUN    TestRestartReplay4A
Test Restart (reliable network)...
... Passed -- time 28.4s #peers 3 #RPCs 467 #Ops 101
--- PASS: TestRestartReplay4A (28.79s)
=== RUN    TestShutdown4A
Test Shutdown (reliable network)...
... Passed -- time 10.0s #peers 3 #RPCs 0 #Ops 0
--- PASS: TestShutdown4A (10.38s)
=== RUN    TestRestartSubmit4A
Test Restart and submit (reliable network)...
... Passed -- time 39.8s #peers 3 #RPCs 463 #Ops 102
--- PASS: TestRestartSubmit4A (40.21s)
PASS
ok      6.5840/kvraft1/rsm 93.691s
```

- You should not need to add any fields to the Raft ApplyMsg, or to Raft RPCs such as AppendEntries, but you are allowed to do so.
- Your solution needs to handle an rsm leader that has called Start() for a request submitted with Submit() but loses its leadership before the request is committed to the log. One way to do this is for the rsm to detect that it has lost leadership, by noticing that Raft's term has changed or a different request has appeared at the index returned by Start(), and return rpc.ErrWrongLeader from Submit(). If the ex-leader is partitioned by itself, it won't know about new leaders; but any client in the same partition won't be able to talk to a new leader either, so it's OK in this case for the server to wait indefinitely until the partition heals.

Part B: Key/value service without snapshots

```
$ cd src
$ make RUN="-run 4B" kvraft1
go build -race -o main/kvraft1d main/kvraft1d.go
cd kvraft1 && go test -v -race -run 4B
=== RUN    TestBasic4B
Test: one client (4B basic) (reliable network)...
Fatal: Wrong error
```

Now you will use the rsm package to replicate a key/value server. Each of the servers ("kvservers") will have an associated rsm/Raft peer. Clerks send Put() and Get() RPCs to the kvserver whose associated Raft is the leader. The kvserver code submits the Put/Get operation to rsm, which replicates it using Raft and invokes your server's DoOp at each peer, which should apply the operations to the peer's key/value database; the intent is for the servers to maintain identical replicas of the key/value database.

A Clerk sometimes doesn't know which kvserver is the Raft leader. If the Clerk sends an RPC to the wrong kvserver, or if it cannot reach the kvserver, the Clerk should re-try by sending to a different kvserver. If the key/value service commits the operation to its Raft log (and hence applies the operation to the key/value state machine), the leader reports the result to the Clerk by responding to its RPC. If the operation failed to commit (for example, if the leader was replaced), the server reports an error, and the Clerk retries with a different server.

Your first task is to implement a solution that works when there are no dropped messages, and no failed servers.

Feel free to copy your client code from Lab 2 (kvsrv1/client.go) into kvraft1/client.go. You will need to add logic for deciding which kvserver to send each RPC to.

You'll also need to implement Put() and Get() RPC handlers in server.go. These handlers should submit the request to Raft using rsm.Submit(). As the rsm package reads commands from applyCh, it should invoke the DoOp method, which you will have to implement in server.go.

You have completed this task when you **reliably** pass the first test in the test suite, with make RUN="-run TestBasic4B" kvraft1.

- A kvserver should not complete a Get() RPC if it is not part of a majority (so that it does not serve stale data). A simple solution is to enter every Get() (as well as each Put()) in the Raft log using Submit(). You don't have to implement the optimization for read-only operations that is described in Section 8.
- It's best to add locking from the start because the need to avoid deadlocks sometimes affects overall code design. The tester runs your code with the race detector by default.

Now you should modify your solution to continue in the face of network and server failures. One problem you'll face is that a Clerk may have to send an RPC multiple times until it finds a kvserver that replies positively. If a leader fails just after committing an entry to the Raft log, the Clerk may not receive a reply, and thus may re-send the request to another leader. Each call to Clerk.Put() should result in just a single execution for a particular version number.

Add code to handle failures. Your Clerk can use a similar retry plan as in lab 2, including returning ErrMaybe if a response to a retried Put RPC is lost. You are done when your code reliably passes all the

4B tests, with make RUN="-run 4B" kvraft1.

- Recall that the rsm leader may lose its leadership and return `rpc.ErrWrongLeader` from `Submit()`. In this case you should arrange for the Clerk to re-send the request to other servers until it finds the new leader.
- You will probably have to modify your Clerk to remember which server turned out to be the leader for the last RPC, and send the next RPC to that server first. This will avoid wasting time searching for the leader on every RPC, which may help you pass some of the tests quickly enough.

Your code should now pass the Lab 4B tests, like this:

```
$ cd src
$ make RUN="-run 4B" kvraft1
go build -race -o main/kvraft1d main/kvraft1d.go
cd kvraft1 && go test -v -race -run 4B
=== RUN    TestBasic4B
Test: one client (4B basic) (reliable network)...
... Passed -- time 3.5s #peers 5 #RPCs 395 #Ops 122
--- PASS: TestBasic4B (4.11s)
=== RUN    TestSpeed4B
Test: one client (4B speed) (reliable network)...
... Passed -- time 33.4s #peers 3 #RPCs 3291 #Ops 1002
--- PASS: TestSpeed4B (33.80s)
=== RUN    TestConcurrent4B
Test: many clients (4B many clients) (reliable network)...
... Passed -- time 4.1s #peers 5 #RPCs 953 #Ops 558
--- PASS: TestConcurrent4B (4.69s)
=== RUN    TestUnreliable4B
Test: many clients (4B many clients) (unreliable network)...
... Passed -- time 4.6s #peers 5 #RPCs 685 #Ops 210
--- PASS: TestUnreliable4B (5.22s)
=== RUN    TestOnePartition4B
Test: one client (4B progress in majority) (unreliable network)...
... Passed -- time 4.9s #peers 5 #RPCs 231 #Ops 4
Test: no progress in minority (4B) (unreliable network)...
... Passed -- time 1.8s #peers 5 #RPCs 110 #Ops 7
Test: completion after heal (4B) (unreliable network)...
... Passed -- time 1.1s #peers 5 #RPCs 43 #Ops 4
--- PASS: TestOnePartition4B (8.36s)
=== RUN    TestManyPartitionsOneClient4B
Test: partitions, one client (4B partitions, one client) (reliable network)...
... Passed -- time 9.4s #peers 5 #RPCs 520 #Ops 114
--- PASS: TestManyPartitionsOneClient4B (10.08s)
=== RUN    TestManyPartitionsManyClients4B
Test: partitions, many clients (4B partitions, many clients (4B)) (reliable
network)...
... Passed -- time 16.1s #peers 5 #RPCs 1271 #Ops 558
--- PASS: TestManyPartitionsManyClients4B (16.68s)
=== RUN    TestPersistOneClient4B
Test: restarts, one client (4B restarts, one client 4B ) (reliable network)...
... Passed -- time 8.4s #peers 5 #RPCs 311 #Ops 62
--- PASS: TestPersistOneClient4B (9.01s)
=== RUN    TestPersistConcurrent4B
Test: restarts, many clients (4B restarts, many clients) (reliable network)...
... Passed -- time 8.5s #peers 5 #RPCs 994 #Ops 350
--- PASS: TestPersistConcurrent4B (9.11s)
=== RUN    TestPersistConcurrentUnreliable4B
Test: restarts, many clients (4B restarts, many clients ) (unreliable network)...
```

```

... Passed -- time 10.3s #peers 5 #RPCs 672 #Ops 114
--- PASS: TestPersistConcurrentUnreliable4B (10.89s)
=== RUN TestPersistPartition4B
Test: restarts, partitions, many clients (4B restarts, partitions, many clients)
(reliable network)...
... Passed -- time 14.3s #peers 5 #RPCs 804 #Ops 94
--- PASS: TestPersistPartition4B (14.95s)
=== RUN TestPersistPartitionUnreliable4B
Test: restarts, partitions, many clients (4B restarts, partitions, many clients)
(unreliable network)...
... Passed -- time 22.0s #peers 5 #RPCs 1229 #Ops 102
--- PASS: TestPersistPartitionUnreliable4B (22.64s)
=== RUN TestPersistPartitionUnreliableLinearizable4B
Test: restarts, partitions, random keys, many clients (4B restarts, partitions, random
keys, many clients) (unreliable network)...
... Passed -- time 24.1s #peers 7 #RPCs 4464 #Ops 444
--- PASS: TestPersistPartitionUnreliableLinearizable4B (24.94s)
PASS
ok 6.5840/kvraft1 175.518s

```

The numbers after each Passed are real time in seconds, number of peers, number of RPCs sent (including client RPCs), and number of key/value operations executed (Clerk Get/Put calls).

Part C: Key/value service with snapshots

As things stand now, your key/value server doesn't call your Raft library's Snapshot() method, so a rebooting server has to replay the complete persisted Raft log in order to restore its state. Now you'll modify kvserver and rsm to cooperate with Raft to save log space and reduce restart time, using Raft's Snapshot() from Lab 3D.

The tester passes maxraftstate to your StartKVServer(), which passes it to rsm. maxraftstate indicates the maximum allowed size of your persistent Raft state in bytes (including the log, but not including snapshots). You should compare maxraftstate to rf.PersistBytes(). Whenever your rsm detects that the Raft state size is approaching this threshold, it should save a snapshot by calling Raft's Snapshot. rsm can create this snapshot by calling the Snapshot method of the StateMachine interface to obtain a snapshot of the kvserver. If maxraftstate is -1, you do not have to snapshot. The maxraftstate limit applies to the GOB-encoded bytes your Raft passes as the first argument to persister.Save().

You can find the source for the persister object in tester1/persister.go.

Modify your rsm so that it detects when the persisted Raft state grows too large, and then hands a snapshot to Raft. When a rsm server restarts, it should read the snapshot with persister.ReadSnapshot() and, if the snapshot's length is greater than zero, pass the snapshot to the StateMachine's Restore() method. You complete this task if you pass TestSnapshot4C in rsm.

```

$ cd src
$ make RUN="-run 4C" kvraft1
go build -race -o main/kvraft1d main/kvraft1d.go
cd kvraft1 && go test -v -race -run 4C
=== RUN TestSnapshotRPC4C
Test: snapshots, one client (4C SnapshotsRPC) (reliable network)...
Test: InstallSnapshot RPC (4C) (reliable network)...
signal: killed
FAIL 6.5840/kvraft1 61.186s

```

- Think about when rsm should snapshot its state and what should be included in the snapshot beyond just the server state. Raft stores each snapshot in the persister object using `Save()`, along with corresponding Raft state. You can read the latest stored snapshot using `ReadSnapshot()`.
- Capitalize all fields of structures stored in the snapshot.

Implement the `kvraft1/server.go` `Snapshot()` and `Restore()` methods, which rsm calls. Modify rsm to handle `applyCh` messages that contain snapshots.

- You may have bugs in your Raft and rsm library that this task exposes. If you make changes to your Raft implementation make sure it continues to pass all of the Lab 3 tests.
- A reasonable amount of time to take for the Lab 4 tests is 400 seconds of real time and 700 seconds of CPU time.

Your code should pass the 4C tests (as in the example here) as well as the 4A+B tests (and your Raft must continue to pass the Lab 3 tests).

```
$ make RUN="-run 4C" kvraft1
go build -race -o main/kvraft1d main/kvraft1d.go
cd kvraft1 && go test -v -race -run 4C
=== RUN    TestSnapshotRPC4C
Test: snapshots, one client (4C SnapshotsRPC) (reliable network)...
Test: InstallSnapshot RPC (4C) (reliable network)...
... Passed -- time 4.8s #peers 3 #RPCs 248 #Ops 72
--- PASS: TestSnapshotRPC4C (5.18s)
=== RUN    TestSnapshotSize4C
Test: snapshots, one client (4C snapshot size is reasonable) (reliable network)...
... Passed -- time 21.0s #peers 3 #RPCs 2569 #Ops 1200
--- PASS: TestSnapshotSize4C (21.42s)
=== RUN    TestSpeed4C
Test: snapshots, one client (4C speed) (reliable network)...
... Passed -- time 24.9s #peers 3 #RPCs 3208 #Ops 1002
--- PASS: TestSpeed4C (25.32s)
=== RUN    TestSnapshotRecover4C
Test: restarts, snapshots, one client (4C restarts, snapshots, one client) (reliable network)...
... Passed -- time 8.2s #peers 5 #RPCs 273 #Ops 50
--- PASS: TestSnapshotRecover4C (8.78s)
=== RUN    TestSnapshotRecoverManyClients4C
Test: restarts, snapshots, many clients (4C restarts, snapshots, many clients ) (reliable network)...
info: linearizability check timed out, assuming history is ok
info: linearizability check timed out, assuming history is ok
info: linearizability check timed out, assuming history is ok
... Passed -- time 12.5s #peers 5 #RPCs 3525 #Ops 1670
--- PASS: TestSnapshotRecoverManyClients4C (13.15s)
=== RUN    TestSnapshotUnreliable4C
Test: snapshots, many clients (4C unreliable net, snapshots, many clients) (unreliable network)...
... Passed -- time 5.5s #peers 5 #RPCs 773 #Ops 230
--- PASS: TestSnapshotUnreliable4C (6.16s)
=== RUN    TestSnapshotUnreliableRecover4C
Test: restarts, snapshots, many clients (4C unreliable net, restarts, snapshots, many clients) (unreliable network)...
... Passed -- time 10.7s #peers 5 #RPCs 804 #Ops 78
--- PASS: TestSnapshotUnreliableRecover4C (11.28s)
=== RUN    TestSnapshotUnreliableRecoverConcurrentPartition4C
```

```
Test: restarts, partitions, snapshots, many clients (4C unreliable net, restarts,
partitions, snapshots, many clients) (unreliable network)...
... Passed -- time 17.4s #peers 5 #RPCs 894 #Ops 94
--- PASS: TestSnapshotUnreliableRecoverConcurrentPartition4C (17.97s)
=== RUN TestSnapshotUnreliableRecoverConcurrentPartitionLinearizable4C
Test: restarts, partitions, snapshots, random keys, many clients (4C unreliable net,
restarts, partitions, snapshots, random keys, many clients) (unreliable network)...
... Passed -- time 19.6s #peers 7 #RPCs 2957 #Ops 368
--- PASS: TestSnapshotUnreliableRecoverConcurrentPartitionLinearizable4C (20.45s)
PASS
ok 6.5840/kvraft1 130.724s
```